

Parallel A* Password Recovery

Sequential, Pthreads, OpenMP, OpenCilk, CUDA and Hybrid CPU+GPU Implementations

Parallel & Distributed Systems (Homework 4)
Apostolos Pantazis (10910)

October 2026

Abstract

We implement an MD5 password-recovery tool as a memory-bounded **A* state-space search** over a prefix tree and parallelize it across five paradigms: POSIX threads, OpenMP, OpenCilk, CUDA, and a Hybrid Pthreads + CUDA design. We study how wall-clock time scales with the size of the problem — both with the password length L and with the volume of candidates the search must process — on a 28-core host with an NVIDIA RTX 4070 Ti SUPER. Time grows essentially linearly with the number of candidates processed and multiplies by roughly the alphabet size for each additional character position. A single-threaded search cannot recover a length-5 target within the 60s budget, but every parallel variant does once given enough workers (Pthreads from 2 threads, OpenMP/OpenCilk from 4; pure CUDA completes at every tested batch size but is slowest), with the Hybrid CPU+GPU design the fastest overall.

1 Problem Formulation

Given the MD5 digest of an unknown password of known length L , we recover the plaintext. Because cryptographic hashes exhibit the avalanche effect, there is no continuous “distance” between a partial candidate and the target, so the hash does not guide the search. Instead we treat the problem as a graph search over a prefix tree and use A* as a systematic, memory-bounded traversal.

State-space model. **State:** a candidate prefix s at depth d . **Alphabet:** a fixed 76-symbol set (lowercase, uppercase, digits, and `!@#$%^&*()-_+=`). **Path cost:** $g(n) = d \cdot \text{EDGE_COST}$ with $\text{EDGE_COST} = 1$. **Heuristic:** $h(n) = (L - d) \cdot \text{EDGE_COST}$ (remaining depth; admissible and consistent). **Evaluation:** $f(n) = g(n) + h(n) = L$, uniform across all candidates of the target length, so A* reduces to a systematic frontier expansion steered toward leaves. **Goal test:** MD5 verification only when a candidate reaches depth L . To bound memory we apply **beam trimming**: when the queue exceeds $\text{BEAM_CAP} = 3,000,000$, a **quickselect** keeps the best $\text{BEAM_KEEP} = 1,200,000$ in expected $O(n)$ and re-heapifies.

Problem size. A length- L password is a point in a discrete space of L coordinates, each drawn from the 76-symbol alphabet, so the search explores up to 76^L candidates: $76^4 \approx 3.34 \times 10^7$ at length 4 and $76^5 \approx 2.54 \times 10^9$ at length 5. The number actually processed before recovery — expanded nodes plus leaf verifications — varies by target, since A* reaches different leaves at different times. The two quantities that drive cost are therefore the password length and the candidate volume, and adding one character multiplies the worst case by 76; we report runtime against both below.

2 Implementations (brief)

All variants share a binary min-heap priority queue (**push/pop** in $O(\log n)$) with **quickselect** beam-trim, and a dependency-free RFC 1321 MD5. The root space is decomposed into $76^2 = 5,776$ two-character prefix tasks for $L \geq 2$.

- **Pthreads:** workers claim tasks via an atomic counter (`atomic_fetch_add`); each owns a private heap (no lock contention); an atomic flag ends the search on a hit.
- **OpenMP:** same decomposition via `#pragma omp parallel for schedule(dynamic,1)` with pre-allocated per-thread heaps.

- **OpenCilk:** `cilk_for` exposes the sub-tasks to the work-stealing runtime; worker count set by `CILK_NWORKERS`.
- **CUDA:** the CPU drives traversal/heap while completed leaves accumulate in a batch buffer; when full, the batch is hashed in parallel on the GPU. Here `-threads` is the GPU *batch size*.
- **Hybrid Pthreads + CUDA:** each POSIX worker owns a CUDA stream, pinned host memory, and private device buffers/flags — CPU task distribution overlapped with GPU batch hashing.

Setup. 28 logical cores (WSL2, Linux 6.6); NVIDIA RTX 4070 Ti SUPER (`sm_89`); `gcc -O2`, OpenCilk `clang`, `nvcc -O3`. Budgets: `TIME_BUDGET = 60s`, `NODE_BUDGET =  $3.5 \times 10^7$` . Timings are wall-clock for the A* phase. Node counts vary slightly with thread count because the parallel frontier is nondeterministic; this does not affect correctness.

3 Execution Time vs. Number of Data Points

At a fixed length, different targets are recovered after a different number of expanded nodes. Figure 1 plots execution time against the number of data points processed, and the relationship is essentially **linear**: cost is dominated by how many candidates the search must touch, not by any per-candidate complexity. Both the sequential baseline (length 4) and the Hybrid design (length 5, 16 threads) trace straight lines.

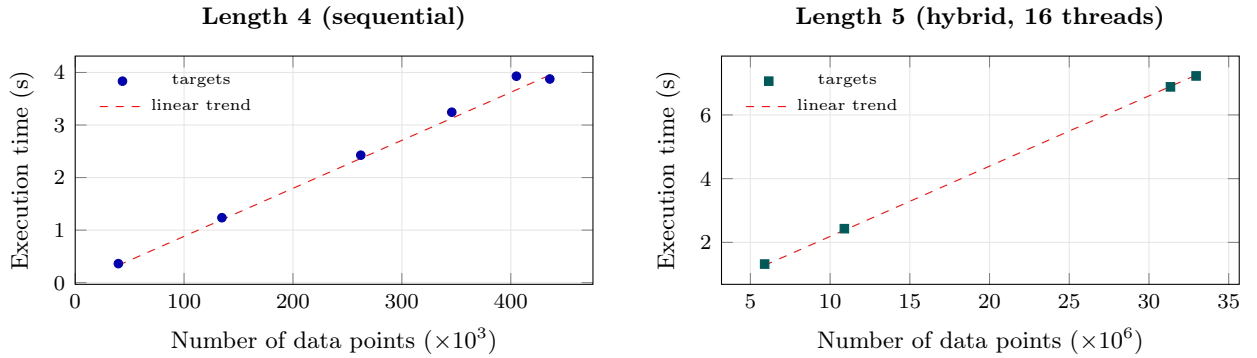


Figure 1: Execution time grows linearly with the number of data points (nodes) processed, at both lengths. Left: six length-4 targets, sequential. Right: four length-5 targets, Hybrid at 16 threads (batch 65,536).

CPU scaling at length 4. Table 1 gives representative times. Adding threads distributes the work across cores; Pthreads and OpenMP scale near-linearly, while OpenCilk’s work-stealing reorders leaf exploration, making it fast on some targets and slower on others.

Table 1: Wall-clock time (s) at length 4 (search space 76^4). *seq* single-threaded; parallel columns use the stated thread count; CUDA uses batch 262,144.

Target	seq	pth-8	pth-32	omp-8	omp-32	cilk-8	cuda
PtYg	0.362	0.253	0.097	0.593	0.200	0.162	0.130
j&mU	1.236	0.060	0.027	0.145	0.051	0.021	0.423
=h#B	2.426	0.483	0.184	1.029	0.360	0.383	0.891
el31	3.244	0.027	0.012	0.062	0.022	0.228	1.054
iEl(	3.929	0.054	0.022	0.117	0.041	0.363	1.156
2h-p	3.876	0.342	0.130	0.769	0.249	0.328	1.364

Throughput vs. threads. Table 2 reports Pthreads self-relative speedup for PtYg: more threads explore the frontier in parallel, giving super-linear speedup at 4–8 threads (the target leaf is reached

out-of-order, earlier than a serial scan), falling to 68% efficiency at 32 threads due to overhead and saturation.

Table 2: Pthreads self-relative speedup, target PtYg, length 4 (baseline 1 thread: 2.106 s).

Thr.	Time	Speedup	Eff.	Thr.	Time	Speedup	Eff.
1	2.106	1.00×	100%	8	0.253	8.32×	104%
2	1.075	1.96×	98%	16	0.150	14.04×	88%
4	0.520	4.05×	101%	32	0.097	21.71×	68%

4 Execution Time vs. Number of Dimensions

Going from length 4 to length 5 adds one dimension and multiplies the worst-case candidate count by $76\times$. The effect on execution time is shown in Figure 2 (note the *logarithmic* time axis). At length 4 every variant recovers the target in well under a second. At length 5 the picture splits by *degree of parallelism*, not by paradigm: the single-threaded Sequential search exhausts the 60s budget, whereas each parallel variant recovers the target once it has enough workers — Pthreads already at 2 threads, OpenMP and OpenCilk from 4, while pure CUDA completes at every tested batch size (slowest of all). The Hybrid design is simply the fastest, overlapping multi-threaded frontier expansion with asynchronous GPU leaf-verification; it is not the only variant that completes length 5. These length-5 numbers are all for target yJuq) (Table 3); completion at length 5 is target-dependent, so a harder target can still drive the low-thread CPU runs to the budget wall.

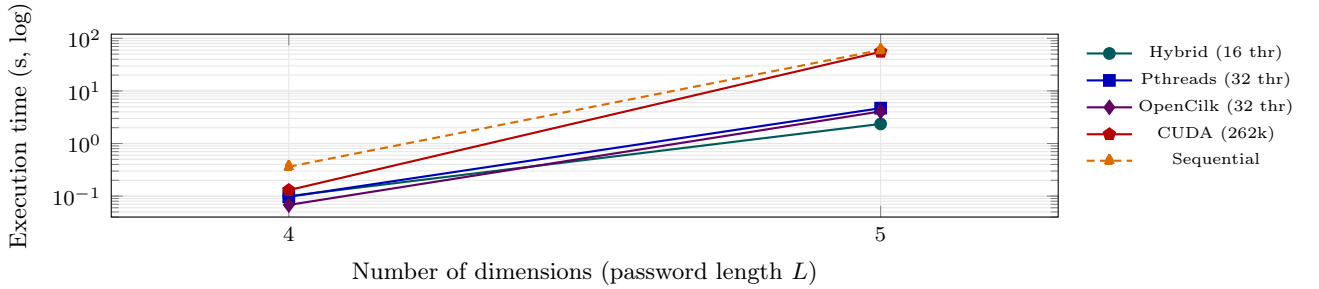


Figure 2: Execution time vs. number of dimensions (log time axis), length-5 target yJuq) (length-4 points use PtYg). One added dimension is a $76\times$ growth in the search space. Only the single-threaded Sequential search hits the 60s budget at length 5 (dashed, plotted at the cap; recovery fails). Every parallel variant recovers the target: Pthreads and OpenCilk in a few seconds at 32 threads, pure CUDA in ≈ 55 s at a 262,144 batch, and the Hybrid design fastest at ≈ 2.4 s (16 threads, 262,144 batch).

Length-5 thread scaling (CPU variants). Table 3 gives the full thread sweep on yJuq). Every variant DNFs at 1 thread, but the CPU-parallel ones recover as soon as a few workers share the search (Pthreads at 2, OpenMP/OpenCilk at 4); time then falls roughly linearly — Pthreads from 49.6 s to 4.7 s ($2 \rightarrow 32$ threads) — with noise from *when* a worker hits the target leaf. Pure CUDA stays slowest and essentially flat across batch sizes ($54.8 \rightarrow 51.9$ s for a $64\times$ larger batch): the serial CPU frontier, not GPU hashing, starves the device, so a bigger batch barely helps.

Table 3: Length-5 wall-clock time (s) on target **yJuq**) (budget 60s; “DNF” = budget exhausted before recovery). Columns are CPU worker threads. Sequential is single-threaded; CUDA and Hybrid are reported below the table since their knob is the GPU batch size.

Variant	1	2	4	8	16	32
Pthreads	DNF	49.59	24.45	12.20	7.50	4.70
OpenMP	DNF	DNF	42.84	26.10	15.10	8.73
OpenCilk	DNF	DNF	22.10	2.92	21.51	4.08

Sequential: DNF (single-threaded, 60.00s cap).
CUDA (GPU batch, all *found*): 65,536 54.76; 262,144 55.39; 1,048,576 52.45; 4,194,304 51.91.
Hybrid (16 thr, batch 262,144): 2.36 — fastest overall (cf. Table 4).

Hybrid at length 5. Table 4 sweeps the GPU batch size at 16 threads for the four length-5 targets. All are recovered. Larger batches amortize kernel-launch overhead over more candidates per launch, cutting both launch count and time; the knee sits around 262,144–1,048,576. At a single thread the Hybrid exhausts the budget just like the other variants; its advantage is that, once threads cooperate, it reaches recovery faster than any CPU-only variant (e.g. 2.4s vs. Pthreads’ 4.7s on **yJuq**)) by overlapping CPU frontier expansion with asynchronous GPU verification.

Table 4: Hybrid batch-size sweep, 16 threads, length 5 (search space $76^5 \approx 2.54 \times 10^9$). All *found*.

Target	Batch	Time (s)	Nodes	Target	Batch	Time (s)	Nodes
yJuq)	16 384	3.056	10 857 302	(K5cq	16 384	8.522	31 350 410
	65 536	2.433	10 880 545		65 536	6.937	31 383 357
	262 144	2.358	10 911 899		262 144	6.814	31 381 106
	1 048 576	2.350	10 994 144		1 048 576	6.922	31 458 800
ntG0y	16 384	1.683	5 896 277	=e!4f	16 384	9.242	32 983 733
	65 536	1.383	5 910 865		65 536	7.394	32 987 203
	262 144	1.315	5 920 023		262 144	7.155	32 953 977
	1 048 576	1.349	6 003 289		1 048 576	7.279	33 120 763

5 Discussion and Conclusion

Candidate volume. Wall-clock time is a near-linear function of how many candidates the search actually processes (Fig. 1); parallelism reduces time by splitting that work across cores, with super-linear speedups when a worker reaches the target leaf early. **Length.** Each added character multiplies the search space by the alphabet size ($76\times$), which is why length 4 is trivial for the CPU but length 5 saturates it (Fig. 2). **Where the GPU pays off.** At length 4 pure CUDA rarely beats a well-parallelized CPU because the bottleneck is CPU-side frontier expansion, not hashing; at length 5 pure CUDA does recover the target but remains the slowest of the completing variants ($\approx 52\text{--}55$ s on **yJuq**)), since its single-threaded frontier, not the hashing, is the limit — which is exactly what the Hybrid design fixes by expanding the frontier on many CPU threads at once. **Batch size.** Small batches (16,384) drown in launch overhead; 262,144–1,048,576 minimize launches per candidate and give the best time. **Oversubscription.** At 32 threads several Hybrid cases show a jump in nodes and time as GPU contention and frontier divergence outweigh added CPU parallelism.

In summary, all six implementations are correct and reproducible. Time scales linearly with candidate volume and multiplicatively with length; at length 4 the CPU variants reach $\approx 21\times$ speedup (Pthreads, 32 threads). At length 5 the decisive factor is the degree of parallelism, not the paradigm: a single thread exhausts the budget while every parallel variant recovers the target, with the Hybrid design fastest (≈ 2.4 s vs. ≈ 4.1 s for the best CPU-only variant on **yJuq**)).

You can find all the necessary files for this project on this GitHub repository:

<https://github.com/pantazisa/Homework/tree/main/Homework%204>